1. Подготовка и инициализация Docker Swarm

В ходе выполнения лабораторной работы использовалась операционная система macOS с установленным Docker Desktop.

Развёртывание стека выполнялось командой:

```
docker stack deploy -c docker-stack.yml web-stack
```

```
[docker stack deploy -c docker-stack.yml web-stack
Since --detach=false was not specified, tasks will be created in the background.
In a future release, --detach=false will become the default.
Creating network web-stack_default
Creating service web-stack_web
```

Проверим состояние

```
[docker service ps web-stack_web
ID             NAME            MODE         REPLICAS   IMAGE                PORT
S
myr2ytku89ry   web-stack_web   replicated   3/3        python:3.11-alpine   *:80
80->5678/tcp
ID             NAME              IMAGE                NODE             DESIRED S
TATE   CURRENT STATE            ERROR     PORTS
wqrupem8rab8   web-stack_web.1   python:3.11-alpine   docker-desktop   Running
       Running 34 seconds ago
j83zcr7khr61   web-stack_web.2   python:3.11-alpine   docker-desktop   Running
       Running 34 seconds ago
kq95kjfid0am   web-stack_web.3   python:3.11-alpine   docker-desktop   Running
       Running 35 seconds ago
```

При первом запуске команда docker stack services показывала 0/3, хотя три задачи уже были созданы. Сначала я подумал, что контейнеры не запустились из-за ошибки, но после проверки docker service ps увидел, что все они находились в состоянии Preparing. Повторная проверка показала, что контейнеры перешли в Running, и сервис начал нормально работать.

После запуска HTTP-сервера была выполнена проверка доступности:

```
[nikita@MacBook-Air-Nikita swarm-lab % curl http://localhost:8080
Hello from Swarm! Container ID: c90e57502ee6 | Slot: 2
```

2. Исследование Routing Mesh и балансировки нагрузки

Для проверки балансировки была выполнена серия из десяти последовательных HTTP-запросов к порту 8080.

Получилось:

```
Request #1:
Hello from Swarm! Container ID: 69b451bfeee3 | Slot: 1
Request #2:
Hello from Swarm! Container ID: 0074e09cd1b9 | Slot: 3
Request #3:
Hello from Swarm! Container ID: c90e57502ee6 | Slot: 2
```

```
Request #4:
Hello from Swarm! Container ID: 69b451bfeee3 | Slot: 1
Request #5:
Hello from Swarm! Container ID: 0074e09cd1b9 | Slot: 3
Request #6:
Hello from Swarm! Container ID: c90e57502ee6 | Slot: 2
Request #7:
Hello from Swarm! Container ID: 69b451bfeee3 | Slot: 1
Request #8:
Hello from Swarm! Container ID: 0074e09cd1b9 | Slot: 3
Request #9:
Hello from Swarm! Container ID: c90e57502ee6 | Slot: 2
Request #10:
Hello from Swarm! Container ID: 69b451bfeee3 | Slot: 1
```

В ходе эксперимента все три реплики участвовали в обработке входящих запросов.

Наблюдалась циклическая последовательность распределения запросов: `1 -> 3 -> 2 -> 1 -> 3 -> 2`.

Такое поведение соответствует алгоритму Round Robin, при котором входящие соединения последовательно распределяются между доступными экземплярами сервиса.

3. Исследование динамического масштабирования Docker Swarm

Для увеличения количества экземпляров приложения до шести использовалась команда:

```
docker service scale web-stack_web=6
```

После выполнения команды было проверено состояние задач:

```
nikita@MacBook-Air-Nikita swarm-lab % docker service ps web-stack_web
ID             NAME              IMAGE                NODE             DESIRED STATE   CURRENT STATE            ERROR     PORTS
wqrupem8rab8   web-stack_web.1   python:3.11-alpine   docker-desktop   Running         Running 8 minutes ago
j83zcr7khr61   web-stack_web.2   python:3.11-alpine   docker-desktop   Running         Running 8 minutes ago
kq95kjfid0am   web-stack_web.3   python:3.11-alpine   docker-desktop   Running         Running 9 minutes ago
isbixkj7m0k6   web-stack_web.4   python:3.11-alpine   docker-desktop   Running         Running 6 seconds ago
s1bkchyoyp2t   web-stack_web.5   python:3.11-alpine   docker-desktop   Running         Running 6 seconds ago
m144h8t1trgd   web-stack_web.6   python:3.11-alpine   docker-desktop   Running         Running 6 seconds ago
nikita@MacBook-Air-Nikita swarm-lab % docker stack services web-stack
ID             NAME            MODE         REPLICAS   IMAGE                PORTS
myr2ytku89ry   web-stack_web   replicated   6/6        python:3.11-alpine   *:8080->5678/tcp
```

Для уменьшения количества реплик с шести до двух использовалась команда:

```
docker service scale web-stack_web=2
```

Сразу после выполнения команды Docker показывал `6/2`, а четыре лишние задачи имели желаемое состояние `Remove`, хотя фактически ещё находились в `Running`. Я проверил состояние сервиса повторно, чтобы убедиться, что Docker завершает удаление лишних контейнеров.

```
ID             NAME              IMAGE                NODE             DESIRED STATE   CURRENT STATE            ERROR     PORTS
wqrupem8rab8   web-stack_web.1   python:3.11-alpine   docker-desktop   Running         Running 9 minutes ago
j83zcr7khr61   web-stack_web.2   python:3.11-alpine   docker-desktop   Running         Running 9 minutes ago
kq95kjfid0am   web-stack_web.3   python:3.11-alpine   docker-desktop   Remove          Running 4 seconds ago
isbixkj7m0k6   web-stack_web.4   python:3.11-alpine   docker-desktop   Remove          Running 4 seconds ago
s1bkchyoyp2t   web-stack_web.5   python:3.11-alpine   docker-desktop   Remove          Running 4 seconds ago
m144h8t1trgd   web-stack_web.6   python:3.11-alpine   docker-desktop   Remove          Running 4 seconds ago
ID             NAME            MODE         REPLICAS   IMAGE                PORTS
myr2ytku89ry   web-stack_web   replicated   6/2        python:3.11-alpine   *:8080->5678/tcp
```

Состояние `Remove` означает, что задачи назначены к удалению, однако процесс их завершения ещё не был отражён в фактическом состоянии на момент проверки.

Спустя время они пропадают:



```
ID             NAME              IMAGE                NODE             DESIRED STATE   CURRENT STATE            ERROR     PORTS
wqrupem8rab8   web-stack_web.1   python:3.11-alpine   docker-desktop   Running         Running 12 minutes ago
j83zcr7khr61   web-stack_web.2   python:3.11-alpine   docker-desktop   Running         Running 12 minutes ago
```

4. Исследование самовосстановления Docker Swarm

Для моделирования сбоя один из работающих контейнеров был принудительно удалён командой:

```
docker rm -f 8a19b921e82
```

После принудительного удаления контейнера третьей реплики Docker Swarm зарегистрировал аварийное завершение задачи:

- ID исходной задачи: `084fwpiy5uzxtugufov5w4o4j`.
- Имя задачи: `web-stack_web.3`.
- Желаемое состояние: `Shutdown`.
- Фактическое состояние: `Failed`.
- Ошибка: `task: non-zero exit (137)`.

Код завершения `137` указывает на завершение процесса сигналом `SIGKILL`, что соответствует выполненному принудительному удалению контейнера.

Для восстановления требуемого количества реплик оркестратор автоматически создал новую задачу:

- ID новой задачи: `f7x8nvnc2oscpi2khunicvhbz`.
- Имя задачи: `web-stack_web.3`.
- Желаемое состояние: `Running`.
- Фактическое состояние: `Running`.
- Ошибки отсутствуют.

При этом две остальные реплики продолжили работу без перезапуска.



```
ID                         NAME                IMAGE
 NODE            DESIRED STATE   CURRENT STATE           ERROR                         PORTS
wqrupem8rab8ge7i5ekrjqmlc   web-stack_web.1       python:3.11-alpine@sha256:d9368b3a5ac59afea7b5d4f2e2aea0941dbf9fdee9c369c5bec00b98244bc929
 docker-desktop   Running         Running 17 minutes ago
j83zcr7khr61ilfoq1dntq586   web-stack_web.2       python:3.11-alpine@sha256:d9368b3a5ac59afea7b5d4f2e2aea0941dbf9fdee9c369c5bec00b98244bc929
 docker-desktop   Running         Running 17 minutes ago
f7x8nvnc2oscpi2khunicvhbz   web-stack_web.3       python:3.11-alpine@sha256:d9368b3a5ac59afea7b5d4f2e2aea0941dbf9fdee9c369c5bec00b98244bc929
 docker-desktop   Running         Running 2 minutes ago
084fwpiy5uzxtugufov5w4o4j    \_ web-stack_web.3   python:3.11-alpine@sha256:d9368b3a5ac59afea7b5d4f2e2aea0941dbf9fdee9c369c5bec00b98244bc929
 docker-desktop   Shutdown        Failed 2 minutes ago     "task: non-zero exit (137)"
```

В итоге количество реплик восстановилось до `3/3` без каких-либо дополнительных действий с моей стороны.

Вывод

На мой взгляд, главное преимущество Docker Swarm перед обычным docker run заключается в том, что не нужно вручную следить за каждым контейнером. При использовании docker run я сам должен запускать контейнеры, контролировать их количество и при необходимости восстанавливать после сбоя.

В Docker Swarm достаточно один раз указать желаемое состояние сервиса, например три работающие реплики. Дальше оркестратор самостоятельно старается поддерживать это состояние. Если один контейнер перестанет работать, Swarm обнаружит проблему и создаст новый.

Также эксперимент показал, что Swarm работает не мгновенно: между желаемым и фактическим состоянием иногда возникает небольшая задержка. Но при этом система самостоятельно выполняет необходимые действия.